

1. CREATE API (POST)

Add new student data

URL

POST <http://localhost:5000/api/students/add>

Code

```
router.post("/add", async (req, res) => {  
  
  try {  
  
    const newStudent = new Student(req.body);  
  
    await newStudent.save();  
  
    res.json({  
      message: "Student Added Successfully",  
      data: newStudent  
    });  
  
  } catch (error) {  
  
    res.status(500).json({  
      error: error.message  
    });  
  
  }  
  
});
```

Postman body

```
{  
  "name": "Tanmay",  
  "email": "tanmay@gmail.com",  
  "age": 21  
}
```

2. READ API (GET)

Fetch all students

URL

GET <http://localhost:5000/api/students>

Code

```
router.get("/", async (req, res) => {  
  
  try {  
  
    const students = await Student.find();  
  
    res.json(students);  
  
  } catch (error) {  
  
    res.status(500).json({  
      error: error.message  
    });  
  
  }  
  
});
```

3. UPDATE API (PUT)

Update existing student data

URL

PUT <http://localhost:5000/api/students/update/:id>

Example

PUT <http://localhost:5000/api/students/update/6861312a3f676b912be3dc4ae>

Code

```
router.put("/update/:id", async (req, res) => {  
  
  try {  
  
    const updatedStudent = await Student.findByIdAndUpdate(  
      req.params.id,  
      req.body,  

```

```

    { new: true }
  );

  res.json({
    message: "Student Updated Successfully",
    data: updatedStudent
  });

} catch (error) {

  res.status(500).json({
    error: error.message
  });

}

});

```

Postman body

```

{
  "name": "Rahul Updated",
  "email": "rahulupdated@gmail.com",
  "age": 25
}

```

4. DELETE API (DELETE)

Delete student data

URL

DELETE <http://localhost:5000/api/students/delete/:id>

Example

DELETE <http://localhost:5000/api/students/delete/6861312a3f676b912be3dc4ae>

Code

```

router.delete("/delete/:id", async (req, res) => {

```

```
try {  
  
    await Student.findByIdAndDelete(req.params.id);  
  
    res.json({  
        message: "Student Deleted Successfully"  
    });  
  
} catch (error) {  
  
    res.status(500).json({  
        error: error.message  
    });  
  
}  
  
});
```

(CRUD) All operation complete code

```
const express = require("express");  
  
const router = express.Router();  
  
const Student = require("../models/Student");  
  
// CREATE API
```

```
router.post("/add", async (req, res) => {

  try {

    const newStudent = new Student(req.body);

    await newStudent.save();

    res.json({
      message: "Student Added Successfully",
      data: newStudent
    });

  } catch (error) {

    res.status(500).json({
      error: error.message
    });

  }

});

// READ API

router.get("/", async (req, res) => {
```

```
try {  
  
  const students = await Student.find();  
  
  res.json(students);  
  
} catch (error) {  
  
  res.status(500).json({  
    error: error.message  
  });  
  
}  
  
});
```

```
// UPDATE API
```

```
router.put("/update/:id", async (req, res) => {  
  
  try {  
  
    const updatedStudent = await Student.findByIdAndUpdate(  
      req.params.id,  
      req.body,  
      { new: true }  
    );  
  }  
});
```

```
);

res.json({
  message: "Student Updated Successfully",
  data: updatedStudent
});

} catch (error) {

  res.status(500).json({
    error: error.message
  });

}

});

// DELETE API

router.delete("/delete/:id", async (req, res) => {

  try {

    await Student.findByIdAndDelete(req.params.id);

    res.json({
      message: "Student Deleted Successfully"
```

```
});

} catch (error) {

    res.status(500).json({
        error: error.message
    });

}

});

module.exports = router;
```

INSTALL BACKEND

STEP 1 — Install Required Software

Install These

1. Node.js

Download from:

[Node.js Official Website](https://nodejs.org/en/)

After install check:

```
node -v
```

```
npm -v
```

2. MongoDB Compass (GUI for database)

Download:

[MongoDB Compass](#)

3. Postman

Download:

[Postman Official Website](#)

STEP 2 — Create Backend Folder

Open VS Code terminal:

```
mkdir backend
```

```
cd backend
```

Initialize Node project:

```
npm init -y
```

STEP 3 — Install Backend Packages

Install Express + MongoDB packages:

```
npm install express mongoose cors dotenv nodemon
```

What These Packages Do

Package	Purpose
express	Creates backend server
mongoose	Connects MongoDB
cors	Connect frontend & backend
dotenv	Stores secret variables
nodemon	Auto restart server

STEP 4 — Create Backend Structure

Create folders/files:

```
backend/  
|  
|— models/  
|   └─ User.js  
|  
|— routes/  
|   └─ userRoutes.js  
|  
|— .env  
|— server.js  
|— package.json
```

creation of frontend

STEP 1 — Create Frontend

Open new terminal:

```
npm create vite@latest frontend
```

STEP 2 — Select React

Choose:

React

Then:

JavaScript

STEP 3 — Open Frontend Folder

```
cd frontend
```

STEP 4 — Install Packages

```
npm install
```

STEP 5 — Install Axios

Axios connects frontend ↔ backend.

```
npm install axios
```

STEP 6 — Run Frontend

```
npm run dev
```

You will get:

<http://localhost:5173>

Frontend is now running 

For running the mernstack project command

For backend

Npm start

For frontend

Npm run dev